

Comprehensive Documentation

Detailed explanation of the AVAILABLE enum fix, root cause analysis, and architecture implications for the delivery-partner-service

Microservices Architecture • Event-Driven Design • State Management

Issue: Cannot resolve symbol 'AVAILABLE'

Service: delivery-partner-service

Component: DeliveryStatus enum (common-lib)

Date: May 28, 2026

Table of Contents

1. Problem Overview
2. Error Details
3. Root Cause Analysis
4. Solution Implementation
5. Technical Context
6. Delivery Partner Lifecycle
7. Architecture Implications
8. Best Practices
9. Testing Considerations
10. Related Patterns

1. Problem Overview

A compilation error occurred in the delivery-partner-service when trying to use a non-existent enum value AVAILABLE from the DeliveryStatus enum.

Error Message

```
Cannot resolve symbol 'AVAILABLE'  
Location: delivery-partner-service/src/main/java/com/fooddelivery/  
delivery/consumer/OrderEventConsumer.java:42
```

Impact

- The OrderEventConsumer class could not compile
- Delivery partner status updates would fail after order delivery
- The messaging flow for marking partners as available was broken

2. Error Details

File: OrderEventConsumer.java

Location: delivery-partner-service/src/main/java/com/fooddelivery/delivery/consumer/OrderEventConsumer.java

Problematic Code (Line 42)

```
case "ORDER_DELIVERED":  
    log.info("Order delivered event received for order: {}, partner: {}",  
        event.getOrderId(), event.getDeliveryPartnerId());  
    // Update delivery partner status to AVAILABLE after delivery  
    updateDeliveryStatus(event.getDeliveryPartnerId(),  
        com.fooddelivery.common.enums.DeliveryStatus.AVAILABLE);  
    break;
```

Error Context

- The code was attempting to set a delivery partner's status to AVAILABLE after completing a delivery
- This is part of the event-driven architecture where order status changes trigger delivery partner status updates
- The enum constant AVAILABLE did not exist in the DeliveryStatus enum

3. Root Cause Analysis

The Missing Enum Value

Original DeliveryStatus Enum:

```
package com.fooddelivery.common.enums;  
  
public enum DeliveryStatus {  
    ASSIGNED,  
    PICKED_UP,  
    OUT_FOR_DELIVERY,  
    DELIVERED  
}
```

Analysis

- The enum only defined statuses for active delivery states
- It was missing the initial state (AVAILABLE) for when partners are free
- The business logic required marking partners as available after delivery completion
- This created a gap in the delivery partner lifecycle management

Why This Was Missed

- The enum was likely designed focusing only on the delivery process itself
- The partner availability state was not considered in the initial design
- As the system evolved, the need to track partner availability became apparent

4. Solution Implementation

Changes Made

File: common-lib/src/main/java/com/fooddelivery/common/enums/DeliveryStatus.java

Updated Enum

```
package com.fooddelivery.common.enums;

public enum DeliveryStatus {
    AVAILABLE,          // NEW: Partner is available for new deliveries
    ASSIGNED,           // Partner has been assigned to an order
    PICKED_UP,          // Partner has picked up the order
    OUT_FOR_DELIVERY,   // Partner is on the way to deliver
    DELIVERED           // Order has been delivered
}
```

What Changed

- Added AVAILABLE as the first enum constant
- Positioned at the beginning to represent the initial/default state
- Maintains logical flow from available → assigned → picked up → out for delivery → delivered

Why This Position?

- AVAILABLE represents the starting state in the partner lifecycle
- Partners start as available, get assigned, then progress through delivery states
- After delivery, they return to AVAILABLE for new assignments
- This creates a complete state machine for delivery partners

5. Technical Context

Microservices Architecture

Service Involved: delivery-partner-service

- Manages delivery partner information and status
- Receives events from order service via RabbitMQ
- Updates partner status based on order lifecycle events
- Communicates with other services through the common library

Common Library

- common-lib contains shared enums, DTOs, and utilities
- DeliveryStatus enum is used across multiple services
- Changes to common enums affect all dependent services

Event-Driven Flow

```
graph TD
    A[Order Service (ORDER_DELIVERED event)] --> B[RabbitMQ (delivery.queue)]
    B --> C[OrderEventConsumer (delivery-partner-service)]
    C --> D[DeliveryPartnerService.updateStatus()]
    D --> E[DeliveryStatus.AVAILABLE]
```

Idempotency Considerations

The OrderEventConsumer includes idempotency handling:

```
if (idempotencyHandler.isMessageProcessed(event.getMessageId())) {
    log.info("Skipping duplicate message: messageId={}",
        event.getMessageId());
    return;
}
```

This ensures that even if the same message is delivered multiple times, the partner status is only updated once, preventing race conditions.

6. Delivery Partner Lifecycle

State Machine

[illegible]



State Transitions

Current State	Event	Next State	Business Meaning
AVAILABLE	Order assigned	ASSIGNED	Partner gets a new delivery
ASSIGNED	Order picked up	PICKED_UP	Partner collects food
PICKED_UP	Start delivery	OUT_FOR_DELIVERY	Partner on the way
OUT_FOR_DELIVERY	Delivery complete	DELIVERED	Order delivered
DELIVERED	Return to pool	AVAILABLE	Partner free for new orders

Business Logic

When Partner Becomes AVAILABLE:

- 1. Order is marked as delivered in order service
- 2. Order service publishes ORDER_DELIVERED event
- 3. Delivery partner service receives event
- 4. Partner status updated to AVAILABLE
- 5. Partner becomes eligible for new order assignments

Why This Matters:

- Ensures partners aren't assigned to multiple orders simultaneously
- Provides real-time visibility into partner availability
- Enables efficient order routing and dispatch
- Prevents overloading partners with too many deliveries

7. Architecture Implications

Shared Enum Design

Principles:

- **Completeness:** Enums should represent all possible states in the lifecycle
- **Logical Ordering:** States should follow the natural flow of the business process
- **Cross-Service Consistency:** Shared enums ensure consistent state representation
- **Future-Proofing:** Design for evolution, anticipate new states

Common Library Impact

- Changes to shared enums require careful consideration
- All services using the enum must be tested
- Database schemas may need updates if enum values are persisted
- API contracts may be affected if enum values are exposed

Service Boundaries

delivery-partner-service Responsibilities:

- Manage delivery partner CRUD operations
- Track partner status and availability
- Handle partner assignment logic
- Respond to order lifecycle events

order-service Responsibilities:

- Manage order lifecycle
- Publish order status change events
- Coordinate with delivery service for assignments

Separation of Concerns:

- Order service doesn't directly update partner status
- Uses event-driven communication to maintain loose coupling
- Each service owns its domain entities and state

Data Consistency

Eventual Consistency:

- Status updates are asynchronous via events
- There may be a brief delay between order delivery and partner availability
- Idempotency handlers prevent duplicate updates
- System is designed to handle temporary inconsistencies

8. Best Practices

Enum Design Guidelines

- **1. Represent Complete State Machine** - Include all possible states in the lifecycle, consider initial, intermediate, and final states
- **2. Logical Ordering** - Order enum values to reflect the natural flow, first value should be the initial/default state

- **3. Meaningful Names** - Use clear, descriptive names, avoid abbreviations, follow naming conventions (UPPER_SNAKE_CASE)
- **4. Documentation** - Document each enum value's purpose, explain state transitions, provide examples of usage

Event-Driven Architecture

- **1. Idempotency** - Always handle duplicate messages, use message IDs for deduplication, design handlers to be safe for retries
- **2. Error Handling** - Log errors comprehensively, implement retry mechanisms, consider dead letter queues for failed messages
- **3. Event Design** - Include all necessary context in events, use consistent event naming, version events for backward compatibility

Microservices Communication

- **1. Shared Libraries** - Use common-lib for shared types, keep common library stable, version shared libraries carefully
- **2. Loose Coupling** - Services communicate via events, not direct calls, each service owns its data, avoid shared databases
- **3. Resilience** - Handle service failures gracefully, implement circuit breakers, use timeouts and retries

9. Testing Considerations

Unit Tests

Test Cases for DeliveryStatus Enum:

```
@Test
void testDeliveryStatusValues() {
    assertEquals(5, DeliveryStatus.values().length);
    assertEquals("AVAILABLE", DeliveryStatus.values()[0].name());
    assertEquals("ASSIGNED", DeliveryStatus.values()[1].name());
    assertEquals("PICKED_UP", DeliveryStatus.values()[2].name());
    assertEquals("OUT_FOR_DELIVERY", DeliveryStatus.values()[3].name());
    assertEquals("DELIVERED", DeliveryStatus.values()[4].name());
}
```

Test Cases for OrderEventConsumer:

```
@Test
void testOrderDeliveredEventUpdatesPartnerToAvailable() {
    OrderEvent event = new OrderEvent();
    event.setEventType("ORDER_DELIVERED");
    event.setDeliveryPartnerId(1L);

    consumer.handleOrderEvent(event);

    verify(deliveryPartnerService).updateStatus(eq(1L),
        any(DeliveryStatusUpdateRequestDto.class));
    // Verify status is AVAILABLE
}
```

Integration Tests

Test Scenarios:

- Complete order flow from creation to delivery
- Partner status transitions through all states
- Duplicate message handling
- Service failure and recovery

Regression Tests

Areas to Verify:

- All services using DeliveryStatus enum
- Database queries filtering by status
- API responses including status
- Event payloads with status values

10. Related Patterns

State Pattern

The delivery partner status could be implemented using the State pattern for more complex behavior:

- Each state encapsulates specific behavior
- State transitions are managed explicitly
- Easy to add new states without modifying existing code

Saga Pattern

For complex multi-step processes:

- Coordinate state changes across services
- Implement compensating transactions
- Ensure consistency in distributed systems

Event Sourcing

Alternative approach for state management:

- Store all state changes as events
- Rebuild current state from event history
- Provides complete audit trail

CQRS

Separate read and write models:

- Optimized queries for partner availability
- Separate data stores for different access patterns
- Improved performance for read-heavy operations

Summary

Problem

- DeliveryStatus.AVAILABLE enum value was missing
- Caused compilation error in OrderEventConsumer
- Broke the delivery partner lifecycle

Solution

- Added AVAILABLE to the DeliveryStatus enum
- Positioned as first value (initial state)
- Completed the partner state machine

Lessons Learned

- Design enums to represent complete state machines
- Consider all lifecycle states during initial design
- Shared types require careful change management
- Event-driven architectures need idempotency

Next Steps

- Update database schemas if status is persisted
- Add comprehensive unit and integration tests
- Update API documentation
- Verify all services using the enum
- Consider adding state transition validation

Delivery Status Enum Fix — Documentation v1.0